CENTRO UNIVERSITÁRIO UniFECAF

Object Oriented Programming

Sistema Automotivo
Gestão de Estoque de Veículos

Sistema CRUD orientado a objetos · Java · Spring Boot · MySQL

Kayke Nogueira de Souza

São Paulo, novembro de 2026

1. Introdução

Concessionárias que controlam o estoque em planilhas ou anotações sofrem com informações desatualizadas: vendedores oferecem carros já vendidos, clientes demoram a encontrar o veículo certo e o gerente não tem uma visão confiável do que está disponível. Para resolver isso, foi desenvolvida uma **API REST em Java com Spring Boot e MySQL** que permite cadastrar marcas, modelos e veículos, consultar o estoque com filtros, atualizar e remover veículos, com uma interface web simples. O projeto aplica os pilares da Programação Orientada a Objetos: encapsulamento, herança, abstração e polimorfismo.

2. Levantamento de requisitos

Os requisitos foram levantados em uma entrevista simulada com o gerente de uma concessionária:

1. **Quem vai usar o sistema no dia a dia?** Vendedores, para consultar e reservar veículos, e o gerente da loja, que cadastra, atualiza preços e acompanha o estoque.
2. **Quais informações de cada veículo são indispensáveis?** Marca, modelo, ano de fabricação, cor, preço, quilometragem, placa e a situação do carro (disponível, reservado, vendido ou fora de linha).
3. **Como o estoque é controlado hoje?** Em planilhas separadas por vendedor e em anotações no quadro da loja. Muitas vezes um vendedor oferece um carro que já foi vendido por outro.
4. **Quais buscas os vendedores mais fazem quando um cliente chega?** Faixa de preço, marca, tipo de carro (SUV, sedan, picape) e ano mínimo. É comum o cliente dizer, por exemplo, que quer um SUV até 120 mil, de 2020 para cima.
5. **Marcas e modelos precisam de cadastro próprio ou podem ser digitados livremente?** Precisam de cadastro próprio. Hoje cada um escreve de um jeito ("VW", "Volks", "Volkswagen"), o que atrapalha as buscas e os relatórios.
6. **O que acontece com o veículo quando ele é vendido?** Ele deve sair da vitrine de disponíveis, mas o histórico precisa continuar existindo. A remoção definitiva só ocorre em casos de erro de cadastro ou descontinuação.
7. **Quais dados mudam com mais frequência?** Preço (promoções e reajustes), quilometragem (test drives e deslocamentos) e status (reservas e vendas).
8. **Existe alguma regra que o sistema deve impedir?** Preço zerado ou negativo, quilometragem que diminui, placa duplicada, ano impossível e vender duas vezes o mesmo carro.
9. **O gerente precisa de algum indicador?** Sim: quantos carros estão disponíveis, reservados e vendidos, e o valor total parado em estoque disponível.
10. **Quantos veículos a loja costuma ter e quantas pessoas usam ao mesmo tempo?** Entre 80 e 300 veículos, com até 10 pessoas consultando ao mesmo tempo. As buscas precisam responder em menos de 1 segundo.

3. Requisitos

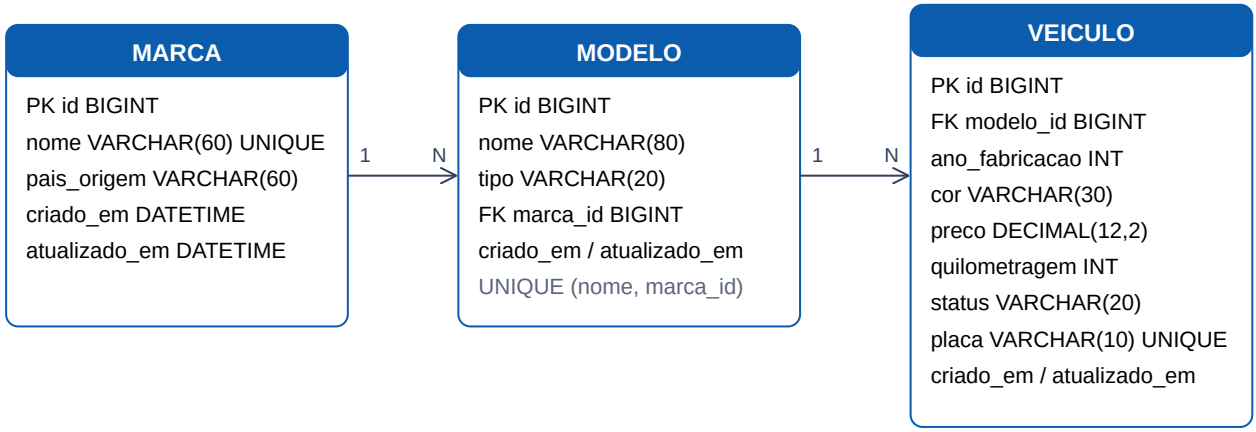
ID	Requisito funcional
RF01	Cadastrar veículos com modelo, marca, ano, cor, preço, quilometragem, placa e status.

RF02	Cadastrar marcas e modelos, sendo cada modelo ligado a uma marca e cada veículo a um modelo.
RF03	Consultar veículos com filtros por marca, modelo, tipo, preço, ano, cor, status e busca livre, com ordenação.
RF04	Atualizar preço, quilometragem e status (ou todos os dados) de um veículo.
RF05	Remover veículos vendidos ou descontinuados, e registrar vendas.

ID	Requisito não funcional
RNF01	**Usabilidade:** interface de uma página, filtros visíveis, formulário em janela e mensagens claras em português.
RNF02	**Performance:** filtros executados no banco com consultas dinâmicas e índices em status, preço e ano.
RNF03	**Integridade:** chaves estrangeiras, campos únicos e validações que impedem dados inválidos.

4. Entidades e modelo de dados

Marca (nome, país de origem), **Modelo** (nome, tipo, marca) e **Veiculo** (modelo, ano, cor, preço, quilometragem, status, placa). Uma marca tem vários modelos e um modelo tem vários veículos. Tipo (HATCH, SEDAN, SUV, PICAPE, MOTO, UTILITARIO) e status (DISPONIVEL, RESERVADO, VENDIDO, DESCONTINUADO) são enums. As três entidades herdam de `EntidadeBase`, que guarda id e datas de criação e atualização.



5. Conceitos de POO aplicados

- **Encapsulamento:** atributos privados e setters que validam antes de alterar. O preço precisa ser positivo, a quilometragem nunca diminui e o ano deve ser válido, então nenhum objeto fica em estado inválido.
- **Herança:** Marca, Modelo e Veiculo estendem `EntidadeBase` e herdam id, datas de auditoria e o preenchimento automático dessas datas.
- **Abstração:** `EntidadeBase` é abstrata e não pode ser instanciada; os repositórios são interfaces cuja implementação o Spring gera.
- **Polimorfismo:** o método abstrato `getDescricao()` é implementado de forma diferente em cada classe ("Toyota (Japao)", "Toyota Corolla - Sedan", "Toyota Corolla 2023 Prata").

Veiculo.java (regra de venda)

```
    public void vender() {
        if (status == StatusVeiculo.VENDIDO) {
            throw new IllegalStateException("Veiculo ja foi vendido");
        }
        this.status = StatusVeiculo.VENDIDO;
    }

    public boolean isDisponivel() {
        return status == StatusVeiculo.DISPONIVEL;
```

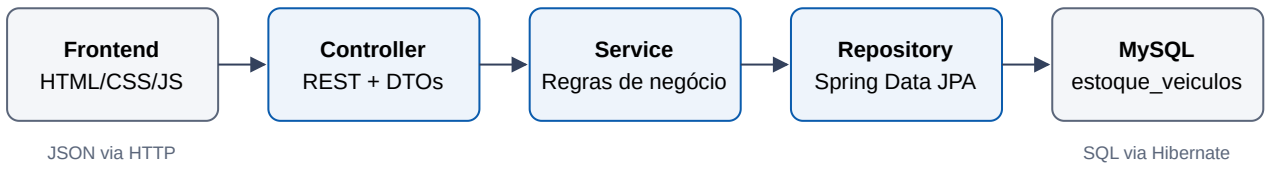
Veiculo.java (encapsulamento da quilometragem)

```
    public void setQuilometragem(Integer novaKm) {
        validarKm(novaKm);
        if (this.quilometragem != null && novaKm < this.quilometragem) {
            throw new IllegalArgumentException("A quilometragem nao pode ser menor que a atual (" +
this.quilometragem + " km)");
        }
        this.quilometragem = novaKm;
    }

    private static Integer validarKm(Integer km) {
        if (km == null || km < 0) {
            throw new IllegalArgumentException("A quilometragem nao pode ser negativa");
        }
        return km;
```

6. Tecnologias e arquitetura

Java 17+ (linguagem orientada a objetos e tipada), **Spring Boot 3** (API REST com configuração automática e servidor embutido), **Spring Data JPA/Hibernate** (mapeamento objeto-relacional e filtros dinâmicos sem SQL manual), **Bean Validation** (validação da entrada), **MySQL 8** (banco relacional com chaves e índices) e **HTML/CSS/JavaScript** no frontend. A arquitetura é em camadas:



Método	Rota	Função
`GET`	`/api/veiculos`	Consulta com filtros (ex.: `?status=DISPONIVEL&precoMax=120000`)
`POST`	`/api/veiculos`	Cadastra veículo
`PUT / PATCH`	`/api/veiculos/{id}`	Atualiza todos os dados / preço, km e status
`POST`	`/api/veiculos/{id}/vender`	Registra venda
`DELETE`	`/api/veiculos/{id}`	Remove veículo
`CRUD`	`/api/marcas`, `/api/modelos`	Cadastro de marcas e modelos

7. Sistema em funcionamento

Prints do sistema rodando localmente em `http://localhost:8080`. Os 6 testes automatizados do projeto (`mvn test`) passaram sem falhas.

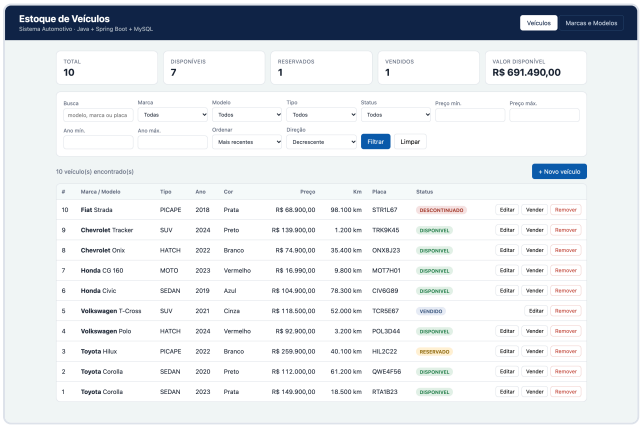


Figura 1. Tela inicial com indicadores e lista do estoque

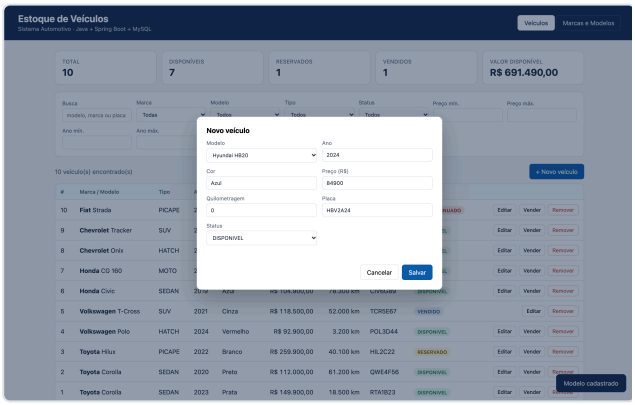


Figura 2. Cadastro de veículo (CREATE)

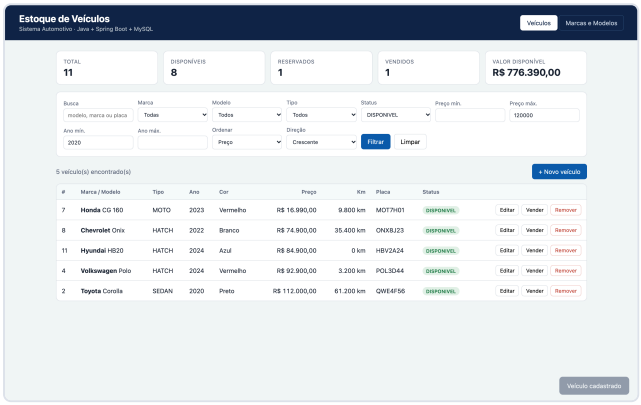


Figura 3. Consulta com filtros: disponíveis, até R$ 120 mil, a partir de 2020 (READ)

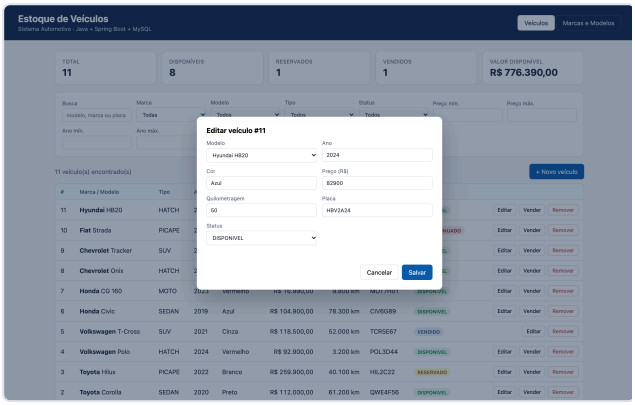


Figura 4. Edição de preço e quilometragem (UPDATE)

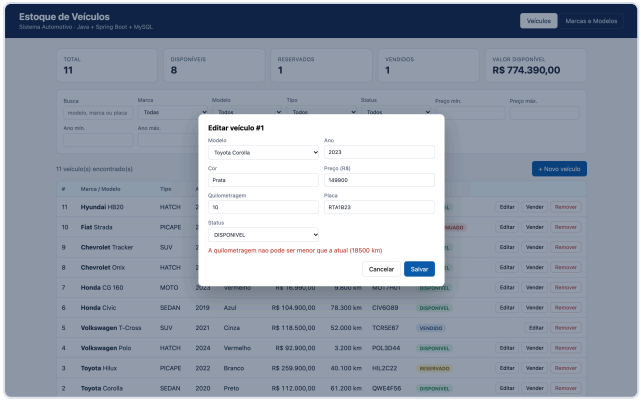


Figura 5. Validação: quilometragem não pode diminuir

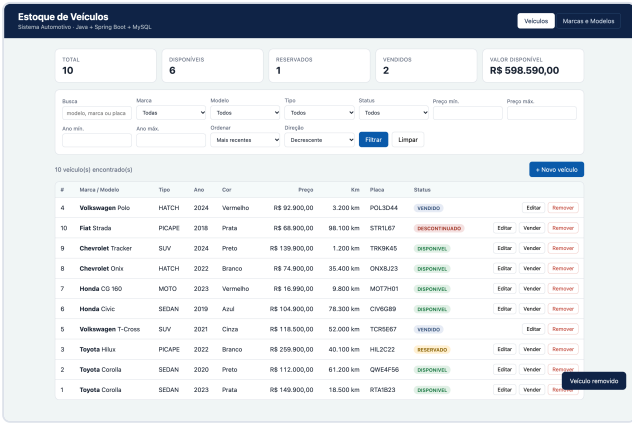


Figura 6. Venda registrada e veículo removido (DELETE)

8. Conclusão

O sistema atende aos requisitos de cadastro, consulta com filtros, atualização e remoção de veículos, com validações que garantem dados consistentes. A aplicação dos conceitos de POO deixou o código mais seguro e fácil de manter, e a separação em camadas permite evoluções como login de usuários, fotos dos veículos e integração com portais de anúncio.

> Código-fonte: **https://github.com/Kayke-ns/estoque-veiculos**

Referências

- DEITEL, P.; DEITEL, H. *Java: como programar*. São Paulo: Pearson.
- SPRING. *Spring Boot Reference Documentation*. https://docs.spring.io/spring-boot/
- ORACLE. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/